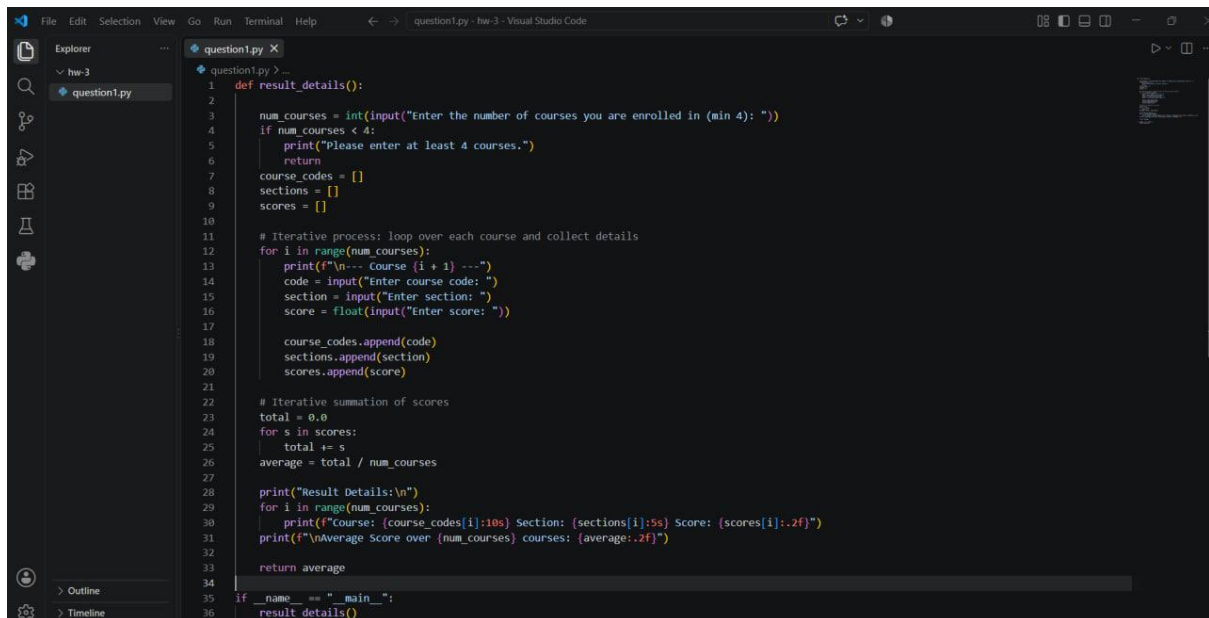


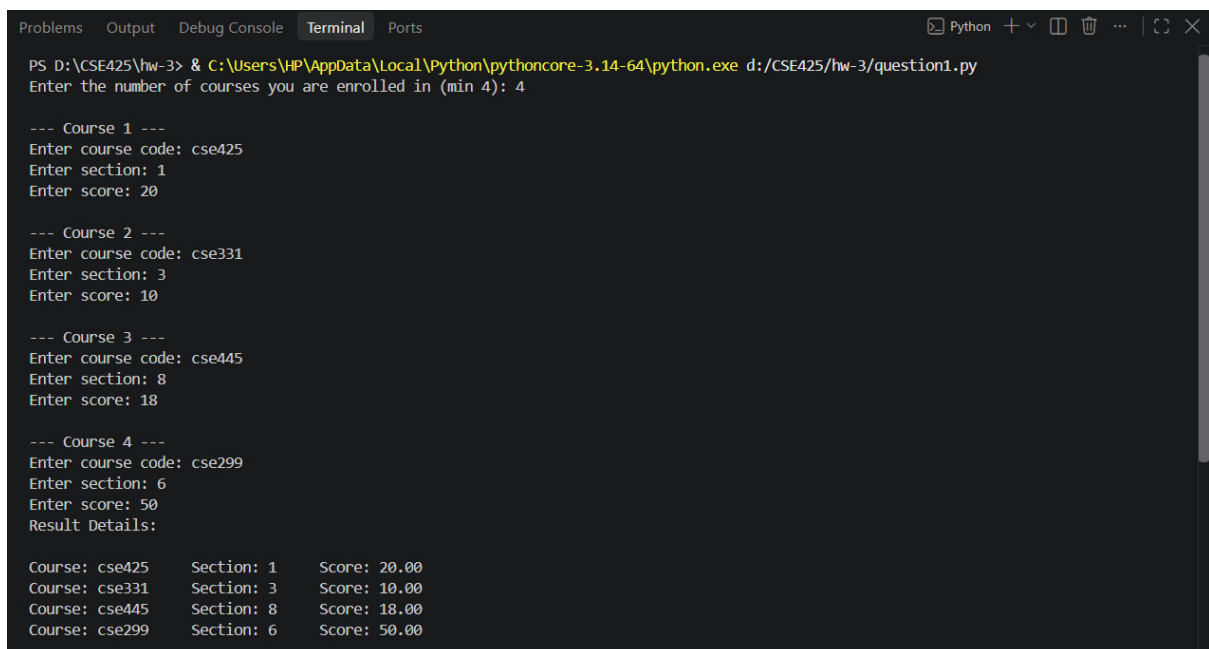
HW – 4

Problem 1 –

In Python, the functions and the whole code, and the output are given below –



```
1 def result_details():
2
3     num_courses = int(input("Enter the number of courses you are enrolled in (min 4): "))
4     if num_courses < 4:
5         print("Please enter at least 4 courses.")
6         return
7     course_codes = []
8     sections = []
9     scores = []
10
11     # Iterative process: loop over each course and collect details
12     for i in range(num_courses):
13         print(f"\n--- Course {i + 1} ---")
14         code = input("Enter course code: ")
15         section = input("Enter section: ")
16         score = float(input("Enter score: "))
17
18         course_codes.append(code)
19         sections.append(section)
20         scores.append(score)
21
22     # Iterative summation of scores
23     total = 0.0
24     for s in scores:
25         total += s
26     average = total / num_courses
27
28     print("Result Details:\n")
29     for i in range(num_courses):
30         print(f"Course: {course_codes[i]:10s} Section: {sections[i]:5s} Score: {scores[i]:.2f}")
31     print(f"\nAverage Score over {num_courses} courses: {average:.2f}")
32
33     return average
34
35 if __name__ == "__main__":
36     result_details()
```



```
PS D:\CSE425\hw-3> & C:\Users\HP\AppData\Local\Python\pythoncore-3.14-64\python.exe d:/CSE425/hw-3/question1.py
Enter the number of courses you are enrolled in (min 4): 4

--- Course 1 ---
Enter course code: cse425
Enter section: 1
Enter score: 20

--- Course 2 ---
Enter course code: cse331
Enter section: 3
Enter score: 10

--- Course 3 ---
Enter course code: cse445
Enter section: 8
Enter score: 18

--- Course 4 ---
Enter course code: cse299
Enter section: 6
Enter score: 50
Result Details:

Course: cse425    Section: 1    Score: 20.00
Course: cse331    Section: 3    Score: 10.00
Course: cse445    Section: 8    Score: 18.00
Course: cse299    Section: 6    Score: 50.00
```

For Fortran, I wrote and ran the code in an online compiler, as my laptop was showing problems while configuring it to run fortran code. Here, the solution for the solution of problem 1 and the output of problem 1 are given below –

```
onlinegdb.com/online_fortran_compiler#

1 program main
2   implicit none
3   call result_details()
4 end program main

5
6 subroutine result_details()
7   implicit none
8   integer :: num_courses, i
9   real :: score, total, average
10  character(len=20), allocatable :: course_codes(:), sections(:)
11  real, allocatable :: scores(:)
12
13  print *, "Enter the number of courses you are enrolled in (min 4):"
14  read *, num_courses
15
16  if (num_courses < 4) then
17    print *, "Please enter at least 4 courses."
18    return
19  end if
20
21  allocate(course_codes(num_courses))
22  allocate(sections(num_courses))
23  allocate(scores(num_courses))
24
25  ! Iterative process: loop over each course and collect details
26  do i = 1, num_courses
27    print *, "--- Course", i, "---"
28    print *, "Enter course code:"
29    read *, course_codes(i)
30    print *, "Enter section:"
31    read *, sections(i)
32    print *, "Enter score:"
33    read *, scores(i)
34  end do
35
36  ! Iterative summation of scores
37  total = 0.0
38  do i = 1, num_courses
39    total = total + scores(i)
40  end do
41  average = total / real(num_courses)
42
43  print *, ""
44  print *, "Result Details"
45  do i = 1, num_courses
46    print *, "Course:", trim(course_codes(i)), " Section:", trim(sections(i)), &
47    " Score:", scores(i)
48  end do
49  print *, ""
50  print *, "Average Score over", num_courses, "courses:", average
51
52  deallocate(course_codes, sections, scores)
53 end subroutine result_details
```

```
onlinegdb.com/online_fortran_compiler#

Enter the number of courses you are enrolled in (min 4):
4
--- Course      1 ---
Enter course code:
cse425
Enter section:
1
Enter score:
20
--- Course      2 ---
Enter course code:
cse331
Enter section:
3
Enter score:
10
--- Course      3 ---
Enter course code:
cse299
Enter section:
6
Enter score:
50
--- Course      4 ---
Enter course code:
cse445
Enter section:
8
Enter score:
18

Result Details
Course:cse425 Section:1 Score: 20.0000000
Course:cse331 Section:3 Score: 10.0000000
Course:cse299 Section:6 Score: 50.0000000
Course:cse445 Section:8 Score: 18.0000000

Average Score over      4 courses: 24.5000000
```

Problem - 2 –

The solution and the output for the problem 2 in Python, is provided below –

The screenshot shows the Visual Studio Code editor with a file named `question2.py` open. The code is a Python script that solves quadratic equations. It uses the `cmath` module for complex roots. The script defines a function `solve_quadratic(a, b, c)` that takes coefficients `a`, `b`, and `c` as input. It first checks if `a` is zero; if so, it prints an error message and returns `None`. Otherwise, it calculates the discriminant $b^2 - 4ac$. If the discriminant is non-negative, it calculates the square root of the discriminant and finds two real roots. If the discriminant is negative, it calculates the square root of the absolute value of the discriminant and finds two complex roots. The `main` function prompts the user to enter coefficients `a`, `b`, and `c`, and then calls `solve_quadratic`. The script is executed using the `if __name__ == '__main__':` guard.

```
1 import cmath
2
3 def solve_quadratic(a, b, c):
4     if a == 0:
5         print("Error: 'a' cannot be zero for a quadratic equation.")
6         return None
7
8     discriminant = b ** 2 - 4 * a * c
9
10    if discriminant >= 0:
11        sqrt_disc = discriminant ** 0.5
12        root1 = (-b + sqrt_disc) / (2 * a)
13        root2 = (-b - sqrt_disc) / (2 * a)
14        print("Roots are real.")
15        print(f"Root 1 = {root1}")
16        print(f"Root 2 = {root2}")
17        return root1, root2
18    else:
19        sqrt_disc = cmath.sqrt(discriminant)
20        root1 = (-b + sqrt_disc) / (2 * a)
21        root2 = (-b - sqrt_disc) / (2 * a)
22        print("Roots are complex")
23        print(f"Root 1 = {root1}")
24        print(f"Root 2 = {root2}")
25        return root1, root2
26
27
28 def main():
29     a = float(input("Enter coefficient a: "))
30     b = float(input("Enter coefficient b: "))
31     c = float(input("Enter coefficient c: "))
32     solve_quadratic(a, b, c)
33
34
35 if __name__ == "__main__":
36     main()
```

The screenshot shows a Windows Command Prompt window with the title bar containing tabs for Problems, Output, Debug Console, Terminal, and Ports. The `Terminal` tab is active. The prompt shows the user running the command `python question2.py` in the directory `D:\CSE425\hw-3`. The program prompts for coefficients `a`, `b`, and `c`, which are entered as 2, 5, and 1 respectively. The program then outputs the roots of the quadratic equation $2x^2 + 5x + 1 = 0$.

```
PS D:\CSE425\hw-3> python question2.py
Enter coefficient a: 2
Enter coefficient b: 5
Enter coefficient c: 1
Roots are real.
Root 1 = -0.21922359359558485
Root 2 = -2.2807764064044154
PS D:\CSE425\hw-3> 
```

In Fortran, the code and the output are provided below –

```
onlinegdb.com/online_fortran_compiler#

1 program main
2   implicit none
3   real :: a, b, c
4
5   print *, "Enter coefficient a:"
6   read *, a
7   print *, "Enter coefficient b:"
8   read *, b
9   print *, "Enter coefficient c:"
10  read *, c
11
12  call solve_quadratic(a, b, c)
13
14 end program main
15
16 subroutine solve_quadratic(a, b, c)
17   implicit none
18   real, intent(in) :: a, b, c
19   real :: discriminant, sqrt_disc, real_part, imag_part
20   real :: root1, root2
21
22   if (a == 0.0) then
23     print *, "Error: 'a' cannot be zero for a quadratic equation."
24     return
25   end if
26
27   discriminant = b**2 - 4.0 * a * c
28
29   if (discriminant >= 0.0) then
30     sqrt_disc = sqrt(discriminant)
31     root1 = (-b + sqrt_disc) / (2.0 * a)
32     root2 = (-b - sqrt_disc) / (2.0 * a)
33     print *, "Roots are real."
34     print *, "Root 1 =", root1
35     print *, "Root 2 =", root2
36   else
37     print *, "Roots are complex"
38     sqrt_disc = sqrt(-discriminant)
39     real_part = -b / (2.0 * a)
40     imag_part = sqrt_disc / (2.0 * a)
41     print *, "Root 1 =", real_part, "+", imag_part, "i"
42     print *, "Root 2 =", real_part, "-", imag_part, "i"
43   end if
44 end subroutine solve_quadratic
```

```
input
Enter coefficient a:
2
Enter coefficient b:
5
Enter coefficient c:
1
Roots are real.
Root 1 = -0.219223619
Root 2 = -2.28077650

...Program finished with exit code 0
Press ENTER to exit console.
```

Problem - 3 –

The solution and the outcome to the problem 3, in Python are provided below –

```

1 def extract_sorted_integers(mixed_list):
2     integers_only = []
3
4     for element in mixed_list:
5         if isinstance(element, int) and not isinstance(element, bool):
6             integers_only.append(element)
7
8     integers_only.sort()
9     return integers_only
10
11
12
13 def main():
14     mixed_list = [5, "apple", 3.14, 2, "banana", 9, 7.5, "cherry", 1, True, -4]
15     sorted_integers = extract_sorted_integers(mixed_list)
16
17     print("Original (input) list :", mixed_list)
18     print("Modified (integers only, sorted) list:", sorted_integers)
19
20
21 if __name__ == "__main__":
22     main()

```

```

PS D:\CSE425\hw-3> python question3.py
Original (input) list : [5, 'apple', 3.14, 2, 'banana', 9, 7.5, 'cherry', 1, True, -4]
Modified (integers only, sorted) list: [-4, 1, 2, 5, 9]
PS D:\CSE425\hw-3>

```

In Fortran, the solution to the problem and output are given below –

```

1 program main
2   implicit none
3   integer, parameter :: n = 8
4   real :: mixed_values(n)
5   integer :: type_flag(n) ! 1 = integer, 2 = float
6   integer, allocatable :: integers_only(:)
7   integer :: i, count_int, temp
8
9   ! Mixed list: (5, 3.14, 2, 9.8, 9, 7.5, 1, -4)
10  mixed_values = (/ 5.0, 3.14, 2.0, 9.8, 9.0, 7.5, 1.0, -4.0 /)
11  type_flag = (/ 1, 2, 1, 2, 1, 2, 1, 1 /)
12
13  ! Print original input list
14  print *, "Original (input) list:"
15  do i = 1, n
16      if (type_flag(i) == 1) then
17          print *, " Integer:", int(mixed_values(i))
18      else
19          print *, " Float :", mixed_values(i)
20      end if
21  end do
22
23  ! Count how many integers there are
24  count_int = 0
25  do i = 1, n
26      if (type_flag(i) == 1) count_int = count_int + 1
27  end do
28
29  allocate(integers_only(count_int))
30  count_int = 0
31  do i = 1, n
32      if (type_flag(i) == 1) then
33          count_int = count_int + 1
34          integers_only(count_int) = int(mixed_values(i))
35      end if
36  end do
37
38  ! Sort the integers in ascending order
39  call bubble_sort(integers_only, count_int)
40
41  print *, ""
42  print *, "Modified (integers only, sorted) list:"
43  print *, integers_only
44
45  deallocate(integers_only)
46
47 end program main

```

```

21 end do
22
23 ! Count how many integers there are
24 count_int = 0
25 do i = 1, n
26     if (type_flag(i) == 1) count_int = count_int + 1
27 end do
28
29 allocate(integers_only(count_int))
30 count_int = 0
31 do i = 1, n
32     if (type_flag(i) == 1) then
33         count_int = count_int + 1
34         integers_only(count_int) = int(mixed_values(i))
35     end if
36 end do
37
38 ! Sort the integers in ascending order
39 call bubble_sort(integers_only, count_int)
40
41 print *, ""
42 print *, "Modified (integers only, sorted) list:"
43 print *, integers_only
44
45 deallocate(integers_only)
46
47 end program main
48
49
50 subroutine bubble_sort(arr, n)
51   implicit none
52   integer, intent(in) :: n
53   integer, intent(inout) :: arr(n)
54   integer :: i, j, temp
55
56   do i = 1, n - 1
57       do j = i, n - i
58           if (arr(j) > arr(j + 1)) then
59               temp = arr(j)
60               arr(j) = arr(j + 1)
61               arr(j + 1) = temp
62           end if
63       end do
64   end do
65
66 end subroutine bubble_sort

```

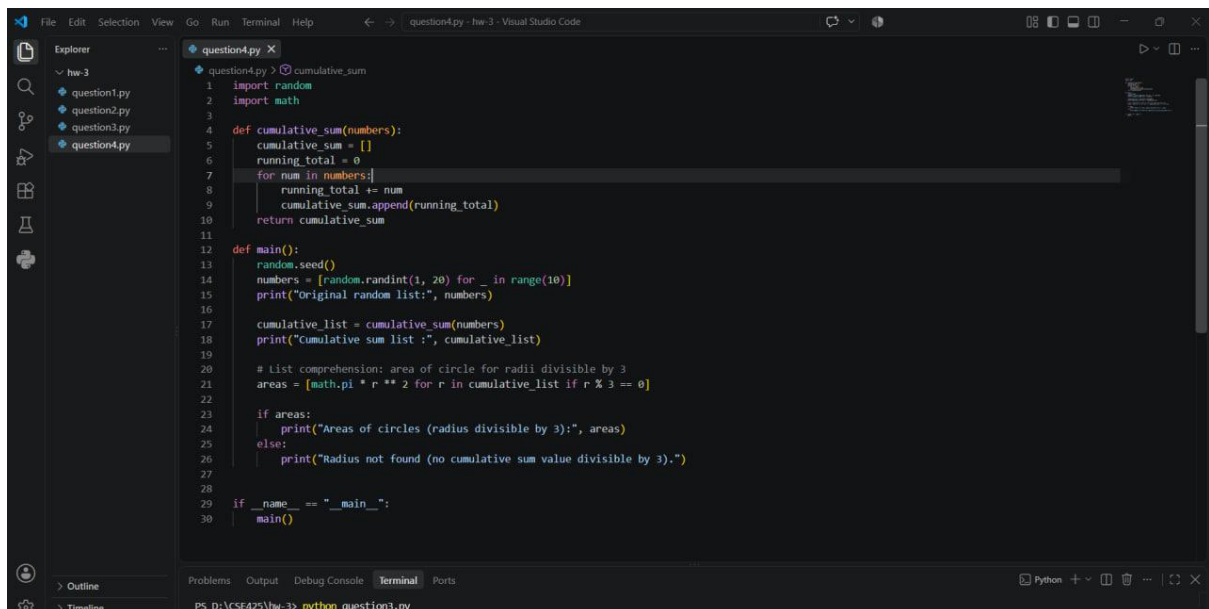
```
Original (input) list:
Integer: 5
Float : 3.14000010
Integer: 2
Float : 9.80000019
Integer: 9
Float : 7.50000000
Integer: 1
Integer: -4

Modified (integers only, sorted) list:
-4      1      2      5      9

...Program finished with exit code 0
Press ENTER to exit console.
```

Problem – 4 –

Here, the solution to the problem and the outcome in python are,



```
question4.py
1 import random
2 import math
3
4 def cumulative_sum(numbers):
5     cumulative_sum = []
6     running_total = 0
7     for num in numbers:
8         running_total += num
9         cumulative_sum.append(running_total)
10    return cumulative_sum
11
12 def main():
13    random.seed()
14    numbers = [random.randint(1, 20) for _ in range(10)]
15    print("Original random list:", numbers)
16
17    cumulative_list = cumulative_sum(numbers)
18    print("Cumulative sum list :", cumulative_list)
19
20    # List comprehension: area of circle for radii divisible by 3
21    areas = [math.pi * r ** 2 for r in cumulative_list if r % 3 == 0]
22
23    if areas:
24        print("Areas of circles (radius divisible by 3):", areas)
25    else:
26        print("Radius not found (no cumulative sum value divisible by 3).")
27
28
29 if __name__ == "__main__":
30    main()
```

PS D:\CSE425\hw-3> python question3.py

```
Problems Output Debug Console Terminal Ports

PS D:\CSE425\hw-3> python question4.py
Original random list: [14, 12, 17, 14, 18, 16, 9, 5, 20, 15]
Cumulative sum list : [14, 26, 43, 57, 75, 91, 100, 105, 125, 140]
Areas of circles (radius divisible by 3): [10207.034531513238, 17671.458676442588, 34636.05900582747]
PS D:\CSE425\hw-3>
```

In Fortran, the code and the output, are given below –

```
onlinegdb.com/online_fortran_compiler#  
main:95  
1 program main  
2 implicit none  
3 integer, parameter :: n = 10  
4 integer :: numbers(n), cumulative_sum(n)  
5 integer :: i, found_count  
6 real :: rand_val  
7 real, parameter :: pi = 3.14159265  
8 real, allocatable :: areas(:)  
9  
10 call random_seed()  
11  
12 ! Step 1: create list of random numbers (1 to 20)  
13 do i = 1, n  
14   call random_number(rand_val)  
15   numbers(i) = int(rand_val * 20) + 1  
16 end do  
17  
18 print *, "Original random list:"  
19 print *, numbers  
20  
21 ! Step 2: cumulative sum  
22 cumulative_sum(1) = numbers(1)  
23 do i = 2, n  
24   cumulative_sum(i) = cumulative_sum(i - 1) + numbers(i)  
25 end do  
26  
27 print *, "Cumulative sum list:"  
28 print *, cumulative_sum  
29  
30 ! Step 3: area of circle for radii divisible by 3  
31 found_count = 0  
32 do i = 1, n  
33   if (mod(cumulative_sum(i), 3) == 0) found_count = found_count + 1  
34 end do  
35  
36 if (found_count == 0) then  
37   print *, "Radius not found (no cumulative sum value divisible by 3)."  
38 else  
39   allocate(areas(found_count))  
40   found_count = 0  
41   do i = 1, n  
42     if (mod(cumulative_sum(i), 3) == 0) then  
43       found_count = found_count + 1  
44       areas(found_count) = pi * real(cumulative_sum(i))**2  
45     end if  
46   end do  
47  
48   print *, "Areas of circles (radius divisible by 3):"  
49   print *, areas  
50  
51   deallocate(areas)  
52 end if  
53  
54 end program main
```

```
input  
Original random list:  
17      2      6      19      2      4      14      16      16      16  
Cumulative sum list:  
17      19      25      44      46      50      64      80      96      112  
Areas of circles (radius divisible by 3):  
28952.9180  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```